Actividad práctica en parejas: proteger un perfil de usuario

Objetivo: transformar una clase que no controla nada en una clase que se hace responsable de sus propios datos, aplicando modificadores de acceso, constructor, this, getters y setters con validación.

Los archivos

El código de la actividad está en el repositorio de la cátedra. Para bajarlo, abrir una terminal en la carpeta donde vayan a trabajar durante el cuatrimestre y clonar el repositorio:

git clone https://github.com/POO-CR/poo-2026.git

También está en archivo comprimido en classroom, dentro hay dos clases:

- PerfilUsuario.java es el primer borrador del sistema. Es el archivo que hay que modificar.
- PruebaPerfil.java es el programa que la usa. No se modifica, salvo para comentar la parte 1 y descomentar la parte 2 cuando el enunciado lo indique.

Parte 1: ver el problema

Abrir los dos archivos y ejecutar PruebaPerfil tal como viene. Mirar la salida en la consola y responder entre los dos, por escrito y en pocas palabras:

1. ¿Qué imprime el programa? ¿Hay algo en esa salida que no tendría que poder existir en un sistema real?
2. El programa no falló ni dio ningún error. ¿Quién tendría que haber impedido que el perfil quedará así?
3. Escribir en una sola frase la promesa de la clase PerfilUsuario: qué tiene que ser cierto siempre para que un perfil tenga sentido.

Parte 2: hacer que la clase se defienda

Refactorizar PerfilUsuario para que sea imposible dejar un perfil en un estado inválido, sin importar lo distraído que esté quien la use. Las reglas que la clase tiene que hacer cumplir son:

- El nombre de usuario no puede ser nulo ni estar vacío, y no cambia nunca una vez creado el perfil.
- El email tiene que contener un arroba. Con esa validación simple alcanza.
- El año de nacimiento no puede ser futuro ni anterior a 1900.
- El nombre completo no puede ser nulo ni estar vacío.

Los pasos están marcados como TODO dentro del archivo. En orden: pasar los atributos a private, escribir el constructor, escribir los getters, escribir solo los setters que tengan sentido y borrar mostrarPerfil().

Tres decisiones importantes, que son el punto de la actividad:

El constructor recibe el nombre de usuario y el email. Si alguno de los dos no sirve, no puede devolver un aviso ni inventar un valor por defecto, así que corta la ejecución con throw new IllegalArgumentException("mensaje claro").

Los setters no imprimen nada. Si el valor es inválido, dejan el estado como estaba y devuelven false. Quien los llama es el que decide cómo avisar, y por eso PruebaPerfil escribe cosas como if (!perfil.setEmail(…)).

No todos los atributos llevan setter. Antes de escribir cada uno hay que poder responder por qué tiene sentido que ese dato cambie desde afuera. Si no hay respuesta, el setter no va.

El método mostrarPerfil() se borra. No porque un método no pueda imprimir: los métodos que muestran datos por consola son cómodos para ver qué está pasando adentro de un objeto, y los vamos a seguir usando durante la materia. Se borra porque acá el que decide qué mostrar es el programa de prueba, y así queda a la vista la diferencia entre las dos responsabilidades que la clase estaba mezclando: validar los datos es del perfil, mostrarlos es de quien lo usa. La distinción importa sobre todo en la validación: un setter que rechaza un valor no puede decidir cómo avisarle al usuario, porque no sabe quién lo está llamando.

Parte 3: verificar y discutir

Comentar la parte 1 de PruebaPerfil y descomentar la parte 2. Antes de ejecutar, conviene notar que la parte 1 ya no compila: eso no es un error, es la prueba de que la refactorización funcionó, porque el programa perdió la posibilidad de escribir los atributos a mano.

El programa tiene que crear un perfil válido, rechazar el email sin arroba y el año futuro sin modificar nada, aceptar los valores buenos y, en la última línea, cortarse con el mensaje del constructor al intentar crear un perfil con el nombre de usuario vacío. Que el programa termine con esa excepción es el resultado esperado.

Para cerrar, discutir en la pareja y anotar las respuestas:

1. ¿Por qué setNombreUsuario no existe, en lugar de existir y devolver siempre false?
2. El constructor corta la ejecución con una excepción, pero los setters devuelven false. ¿Por qué el trato distinto?
3. Un perfil recién creado tiene el nombre completo vacío y el año de nacimiento en cero, porque el constructor no los pide. ¿Es eso un estado válido? Si no lo fuera, ¿qué cambiarían?

Código de partida

```java
// Archivo: PerfilUsuario.java
// Primer borrador de la clase. Funciona, pero no protege nada.

public class PerfilUsuario {

    // Atributos publicos: cualquier parte del programa puede escribir lo que quiera.
    public String nombreUsuario;
    public String nombreCompleto;
    public String email;
    public int anioNacimiento;

    public void mostrarPerfil() {
        System.out.println("--- Perfil ---");
        System.out.println("Usuario: " + this.nombreUsuario);
        System.out.println("Nombre completo: " + this.nombreCompleto);
        System.out.println("Email: " + this.email);
        System.out.println("Anio de nacimiento: " + this.anioNacimiento);
    }

    // TODO 1: pasar los cuatro atributos a private.

    // TODO 2: escribir un constructor que reciba nombreUsuario y email.
    //         Si alguno de los dos es invalido, cortar con
    //         throw new IllegalArgumentException("...");

    // TODO 3: escribir los getters que hagan falta.

    // TODO 4: escribir solo los setters que tengan sentido, devolviendo boolean.
    //         Ningun setter imprime nada: rechaza el valor y avisa con false.

    // TODO 5: borrar mostrarPerfil().
}
```